

# Object Oriented Programming

## Creational Design Patterns: Abstract Factory

IT560



**Sourish Dasgupta**

Assistant Professor, DA-IICT, Gandhinagar

<https://daiict.ac.in/faculty/sourish-dasgupta>

# Relook at Factory Pattern

```
# Base class
class Animal:
    def speak(self):
        raise NotImplementedError("Subclasses must implement this method")
```

```
class Dog(Animal):
    def speak(self):
        return "Woof!"
```

```
class Cat(Animal):
    def speak(self):
        return "Meow!"
```

```
# Factory to create individual animals
```

```
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        else:
            raise ValueError("Unknown")
```

```
# Usage
```

```
dog = AnimalFactory.create_animal("dog")
print(dog.speak()) # Output: Woof!
```

```
cat = AnimalFactory.create_animal("cat")
print(cat.speak()) # Output: Meow!
```

## What's missing??

- Only create **one** type of object (e.g., a **single hierarchy** of animals)
- Doesn't enforce consistency between **related** objects (e.g., "domestic" or "wild" versions of both **Dog** and **Cat**)

# Abstract Factory Design Pattern

```
# Factory to create individual animals
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        else:
            raise ValueError("Unknown animal type")
```



```
# Abstract Factory
class AnimalFactory:
    def create_dog(self):
        raise NotImplementedError("Subclasses must implement this method")

    def create_cat(self):
        raise NotImplementedError("Subclasses must implement this method")
```

# Creating Concrete Factories out of Abstract Factory

```
# Abstract Factory
class AnimalFactory:
    def create_dog(self):
        raise NotImplementedError("Subclasses must implement this method")

    def create_cat(self):
        raise NotImplementedError("Subclasses must implement this method")
```

```
# Concrete Factory 1: Domestic Animals
class DomesticAnimalFactory(AnimalFactory):
    def create_dog(self):
        return DomesticDog()

    def create_cat(self):
        return DomesticCat()
```

```
# Concrete Factory 2: Wild Animals
class WildAnimalFactory(AnimalFactory):
    def create_dog(self):
        return WildDog()

    def create_cat(self):
        return WildCat()
```

# Creating the Classes

```
# Concrete Factory 1: Domestic Animals
class DomesticAnimalFactory(AnimalFactory):
    def create_dog(self):
        return DomesticDog()

    def create_cat(self):
        return DomesticCat()
```

```
# Concrete Factory 2: Wild Animals
class WildAnimalFactory(AnimalFactory):
    def create_dog(self):
        return WildDog()

    def create_cat(self):
        return WildCat()
```

```
# Base Classes
class Animal:
    def speak(self):
        raise NotImplementedError("Subclasses must implement this method")
```

```
class Dog(Animal):
    pass

class Cat(Animal):
    pass
```

```
# Domestic Animals
class DomesticDog(Dog):
    def speak(self):
        return "Woof! (domestic)"

class DomesticCat(Cat):
    def speak(self):
        return "Meow! (domestic)"
```

```
# Wild Animals
class WildDog(Dog):
    def speak(self):
        return "Woof! (wild)"

class WildCat(Cat):
    def speak(self):
        return "Meow! (wild)"
```

# Creating Concrete Factories out of Abstract Factory

```
# Usage
dog = AnimalFactory.create_animal("dog")
print(dog.speak()) # Output: Woof!

cat = AnimalFactory.create_animal("cat")
print(cat.speak()) # Output: Meow!
```



```
# Usage
def animal_interaction(factory: AnimalFactory):
    dog = factory.create_dog()
    cat = factory.create_cat()
    print(dog.speak())
    print(cat.speak())

# Creating domestic animals
domestic_factory = DomesticAnimalFactory()
print("Domestic Animals:")
animal_interaction(domestic_factory)

# Creating wild animals
wild_factory = WildAnimalFactory()
print("\nWild Animals:")
animal_interaction(wild_factory)
```

# Creating Concrete Factories out of Abstract Factory

| Use Case                                                        | Factory Method             | Abstract Factory                                |
|-----------------------------------------------------------------|----------------------------|-------------------------------------------------|
| Single object creation                                          | Works well                 | Overkill                                        |
| <i>Families of related objects (eg. Domestic Animal Family)</i> | Cannot enforce consistency | Enforces consistency across object families     |
| Need for multiple concrete factories                            | Harder to manage           | Each concrete factory handles one family easily |
| Example                                                         | Create a Dog or Cat        | Create a DomesticDog and DomesticCat pair       |